

SIMATS ENGINEERING

Department of Computer Science and Engineering



CSA1512 Cloud Computing and Big Data Analytics

CO2 AT1 - Capstone Infrastructure Sizing Workbook

Guided calibration workbook for VM sizing, virtualization decisions, snapshot planning and VM-container comparison

Assessment Metadata

Course Code	CSA1512
Course Title	Cloud Computing and Big Data Analytics
Assessment	CO2 AT1 - Virtualization Scenario Problem-Solving
Submission Type	Individual digital workbook based on the same capstone title used in CO1
Faculty	Dr. C. Rajesh Babu
Employee ID	SSETSCS-415
Submission Mode	Completed workbook and evidence repository link through Google Classroom

How This Workbook Works

- Read each guidance block and immediately fill the related response table.
- Use the same capstone title selected for CO1.
- Make reasonable assumptions when exact values are unknown. Good assumptions must be written clearly.
- The goal is not to guess a perfect industry configuration; the goal is to reason technically and justify the first infrastructure plan.
- The final decisions from this workbook will be used again in CO2 AT2 and CO2 AT3.

Technical Tip

A good infrastructure estimate is transparent. If a number is assumed, write why it is assumed. If a number is calculated, show the formula. If a service is selected, justify why it fits the workload.

Student and Capstone Details

Student Name	D.S.THARUNIYA PRIYA
Register Number	192511317
Class / Slot	Slot B
Capstone Title	PulsePay – Cloud based Fake payment Screenshot Detection
Capstone Product Type	Web app
CO1 Architecture Reference	Mention concept map / presentation title / GitHub link if available

Activity 1: Bring Forward CO1 Capstone Context

Start with the cloud product idea already used in CO1. The infrastructure plan must support that same application, not a new unrelated example.

Capstone Element	Student Entry
Primary users	Small merchants, online sellers, delivery/gig workers and freelance service providers who receive UPI payment screenshots as proof of payment
User roles	Merchant (uploads screenshot, views verdict); Admin (monitors flagged cases and platform analytics)
Main modules	screenshot upload, OCR extraction, forensic (ELA) tamper analysis, ML fraud-risk scoring, merchant analytics dashboard
Cloud services identified in CO1	Cloud Functions, API Gateway, Firebase Authentication, Cloud Storage, Firestore, Cloud Vision API, Cloud Monitoring
API/backend need	A REST API that accepts an uploaded screenshot, triggers OCR and forensic analysis, and returns a JSON authenticity score with flagged reasons
Database need	A low-latency NoSQL store (Firestore) for verification records, merchant accounts and historical fraud-risk scores
File upload/storage need	Secure object storage (Cloud Storage) for raw screenshot images, with lifecycle rules to auto-delete after a retention window
Analytics/AI/Python module	Python-based OCR (Cloud Vision API), Error Level Analysis (ELA) for tamper detection, and a lightweight ML classifier that combines both signals into a fraud-risk score

Activity 2: Workload Classification

Classify the capstone workload before selecting a VM. A form-based SaaS app and an AI analytics app should not receive the same infrastructure plan.

Workload Type	Typical Signals	Starting VM Guidance
Light web / form app	Login, CRUD, simple reports, low file upload	1-2 vCPU, 2-4 GB RAM
API backend	Authentication, REST APIs, moderate concurrent requests	2 vCPU, 4-8 GB RAM
Database-heavy app	Search, many records, frequent writes	2-4 vCPU, 8-16 GB RAM
File-heavy app	Images, PDFs, uploads, downloads	2 vCPU, 4-8 GB RAM + separate object storage
Analytics app	Dashboards, aggregation, batch reports	4 vCPU, 16 GB RAM
AI/Python module	Prediction, NLP, image/text processing	4-8 vCPU, 16-32 GB RAM; GPU optional
High traffic app	Large user base and peak demand	Multiple VMs or autoscaling group

Technical Tip

Start small for prototype, but do not hide growth. A valid answer can say: prototype uses one VM, pilot separates database, production uses load balancing and managed services.

Student Decision: My capstone workload type and reason

PulsePay is primarily an AI/Python module workload layered on a light API backend. Each request involves OCR and forensic image analysis rather than simple CRUD, so it needs more CPU and memory per request than a plain form app — but traffic per merchant is low-frequency (a few screenshots a day), so it doesn't need constant high-traffic infrastructure. I'm treating it as an "API backend" + "AI/Python module" hybrid: a light API layer for auth/routing (2 vCPU, 4-8 GB) plus a separately-scaled AI/Python service for OCR/ELA/ML scoring (starting around 4 vCPU, 8-16 GB, no GPU needed since ELA and OCR-via-API aren't deep-learning-heavy).

Activity 3: User and Traffic Calibration

Use the following formulas to convert user assumptions into approximate traffic.

Metric	Formula	Example
Concurrent Active Users	Registered Users x Active User % x Peak Factor	$500 \times 10\% \times 2 = 100$
Requests per Minute	Concurrent Active Users x Actions per User per Minute	$100 \times 2 = 200$
Requests per Second	Requests per Minute / 60	$200 / 60 = 3.33 \text{ RPS}$
Daily Requests	Daily Active Users x Avg Actions per Day	$200 \times 30 = 6000$
Input	Value	Reason / Assumption
Registered users expected	1,000 merchants	Realistic pilot size for a first city/region rollout
Active user percentage	10%	Small merchants check payments only when a sale happens, not continuously
Peak factor	2	Festival/sale-season spikes roughly double normal activity
Actions per user per minute	1	One screenshot verification request per active minute
Calculated RPS	3.33 RPS	$(1,000 \times 10\% \times 2) = 200$ concurrent users $\rightarrow 200 \times 1 = 200$ RPM $\rightarrow 200/60 = 3.33 \text{ RPS}$
Daily active users	300	~30% of registered merchants transact on a given day
Daily requests	1,500	300 daily active users $\times 5$ avg verifications/day

Activity 4: CPU Calibration

CPU sizing depends on request rate and processing complexity. Use the score below as a guided estimate.

CPU Driver	Score
Base web/API application	1
Authentication and role management	+1
Moderate API traffic above 5 RPS	+1
Database-heavy search or reporting	+1 to +2

Background jobs or notifications		+1
Analytics or dashboards		+2
AI/Python inference inside server		+2 to +4
Total CPU Score		Recommended vCPU
1-2		1-2 vCPU
3-4		2 vCPU
5-6		4 vCPU
7-9		4-8 vCPU or separate analytics/AI service
10+		Scale out with multiple VMs or managed compute
CPU Driver Selected	Score	Reason
Base web/API application	1	Core Cloud Function handling every request/response
Authentication and roles	+1	Firebase Authentication checks merchant identity on each call
API traffic	0	Peak load (~3.33 RPS) stays below the 5 RPS threshold
Database/search/reporting	+1	Firestore queries for merchant history and repeat-fraud lookups
Background jobs	+1	Scheduled function aggregates daily fraud stats and sends alerts
Analytics/dashboard	+2	Merchant dashboard aggregates risk scores and trends over time
AI/Python module	+2	OCR (Vision API) + ELA run per screenshot; kept lower since Vision API offloads the heaviest inference
Total CPU Score	8	
Recommended vCPU	4-8 vCPU or separate analytics/AI service	Implemented as 2 vCPU for the API layer + a separately scaled 4 vCPU AI/ELA service

Activity 5: RAM Calibration

RAM must include operating system, runtime, application process, database/cache and a safety buffer.

RAM Formula

Estimated RAM = OS/runtime + app/API + database/cache + analytics/AI + 25% buffer. If database is managed separately, do not include full database memory in the app VM.

Component	Guidance	Student Estimate
OS + runtime	1-2 GB	1.5 GB
Web/API service	1-2 GB	1.5 GB
Small database	2-4 GB	Not included — Firestore is fully managed
Medium database/cache	4-8 GB	Not included — Firestore is fully managed
Analytics/reporting	4-8 GB extra	2 GB
AI/Python module	4-16 GB extra	4 GB
25% buffer	Subtotal x 0.25	Subtotal 9 GB × 0.25 = 2.25 GB
Final RAM	Round up to standard size	9 + 2.25 = 11.25 → round up to 12 GB

Activity 6: Storage Calibration

Storage must cover application files, database records, uploads, logs and backup/growth buffer.

Storage Item	Formula / Guidance	Student Estimate
OS and application	20-40 GB	30 GB
Database	Avg record KB x records/day x days x 1.3 / 1024 MB	Not counted in VM disk — Firestore is fully managed with its own auto-scaling storage
File uploads	Avg file MB x files/day x days x 1.3	0.5 MB avg × 1,500 files/day × 30 days × 1.3 ≈ 29.3 GB — held in Cloud Storage, not VM disk
Logs	Requests/day x log KB x days / 1024 MB	1,500 req/day × 2 KB × 30 days / 1024 ≈ 0.09 GB
Backup/growth buffer	Subtotal x 30%	(30 + 0.09) × 30% ≈ 9.03 GB
Final storage	Round up to standard disk size	VM/app disk ≈ 40 GB (rounded); Cloud Storage bucket starts at ~30 GB and auto-grows

Technical Tip

Do not store large uploads permanently inside the VM disk when object storage is available. VM disk is for OS and application. Object storage is better for images, PDFs, media and documents.

Activity 7: Select the VM Plan

Plan	When It Fits	Student Choice
Single VM prototype	Small demo, simple app, low traffic	Used only for early local testing of the OCR/ELA pipeline before cloud deployment
Two-tier design	App VM + managed/separate database	Not primary — PulsePay's compute is stateless and serverless, not a persistent app-VM tier
Three-tier design	Frontend, backend/API and database separated	Reflected logically: Frontend (Firebase Hosting) / Backend (Cloud Functions) / Database

		(Firestore) are separated even without persistent VMs
Containerized backend	Microservices, portable API, CI/CD	Future option for the ELA/ML module if it outgrows Cloud Functions' memory/time limits
Serverless function	Event-based task, notification, small automation	primary compute for OCR, ELA and ML scoring (Cloud Functions)
Managed database/storage	Reliability, backup and scaling required	Firestore for records, Cloud Storage for screenshot images
Final VM Plan: selected plan, vCPU, RAM, storage and reason Serverless-first hybrid: Cloud Functions (≈ 2 vCPU / 4 GB equivalent) for API and auth, scaling to a Cloud Run container (≈ 4 vCPU / 8 GB) for OCR/ELA/ML scoring under heavier load. Firestore and Cloud Storage handle data and files outside the compute tier. For local development, a single prototype VM (2 vCPU, 4 GB RAM, 40 GB disk) under a Type-2 hypervisor is used before any cloud billing begins. This keeps cost near-zero at low merchant volume while giving the AI module a clear scale-up path.		

Activity 8: Hypervisor and Virtualization Decision

Approach	Meaning	Best Use
Type 1 hypervisor	Runs directly on physical hardware	Production data center and cloud provider infrastructure
Type 2 hypervisor	Runs above an existing OS	Student lab, local testing, VirtualBox/VMware Workstation
Cloud-managed virtualization	Cloud provider hides hypervisor details	Most public cloud VM deployments
Container runtime	OS-level process isolation	Lightweight APIs, microservices, fast deployment
Context	Selected Approach	Justification
Local prototype/testing	Type 2 hypervisor (VirtualBox)	Used on a personal laptop to test the OCR/ELA pipeline and Cloud Functions emulator before deploying, with no cloud billing during early development
Cloud pilot deployment	Cloud-managed virtualization	Cloud Functions and Firebase abstract the hypervisor layer entirely, matching a small team's low ops capacity
Production growth scenario	Cloud-managed virtualization + container runtime (hybrid)	As merchant volume grows, the ML scoring module can move to Cloud Run (container runtime) for more control over memory/CPU while the rest stays serverless
Module suitable for container	Container runtime	The ELA/ML scoring module has heavier, longer-running image processing that benefits from predictable container resource allocation over Cloud Functions' execution-time limits

Activity 9: Snapshot and Clone Strategy

Snapshot protects a point in time. Clone creates a duplicate environment. Both are useful, but they are not the same.

Event	Snapshot or Clone?	Frequency / Timing	Retention / Reason
Before major deployment	Snapshot	Before every production push	Quick rollback if new OCR/ELA logic misbehaves
Before database schema change	Snapshot (Firestore export)	Before modifying the verification-records schema	Preserves existing merchant data if migration fails
Before OS/package update	Snapshot	Before updating the dev VM's OS or Python/OpenCV dependencies	Revert quickly if a library update breaks the ELA pipeline
Before review/demo	Snapshot	A few hours before every evaluator demo	Guarantees a known-good state, independent of last-minute changes
Create testing environment	Clone	When testing a new fraud-scoring model version	Run the new model in parallel without affecting the live flow
Create team demo copy	Clone	Before presentations (e.g. Tech Star Summit-style)	A disposable copy that can be reset or broken without risking the main project

Technical Tip

A snapshot is useful for rollback. A clone is useful for parallel testing. A clone should not be treated as the only backup.

Activity 10: VM vs Container Decision

Criteria	Virtual Machine	Container
Isolation	Strong OS-level isolation	Process-level isolation
Startup	Slower	Faster
Resource use	Higher	Lower
Best for	Full OS, legacy app, database VM	Microservices, APIs, portable services
Portability	Moderate	High
Student use	Clear infrastructure learning	Modern deployment learning
Capstone Module	VM / Container / Managed Service	Reason
Frontend	Managed Service (Firebase Hosting)	Static build served via global CDN, no server management needed
Backend/API	Serverless Function (Cloud Functions + API Gateway)	Requests are short, stateless and spiky — ideal for pay-per-invocation compute
Database	Managed Service (Firestore)	Fully managed NoSQL store with built-in scaling and replication
File storage	Managed Service (Cloud Storage)	Purpose-built object storage for screenshots —

		cheaper and more durable than VM disk
AI/Python module	Container (Cloud Run), Cloud Function fallback	ELA/OCR pre-processing can exceed default function memory/time limits under load
Analytics/dashboard	Serverless Function	Aggregation runs periodically via a scheduled function, no always-on server needed
Notification/background jobs	Serverless Function (Cloud Functions + Pub/Sub)	Event-driven alerts fire only when triggered, matching serverless billing

Activity 11: Final Recommendation

Write the final infrastructure recommendation in 8-12 technical lines. Include VM size, hypervisor/virtualization approach, storage plan, snapshot/clone plan and VM-container decision.

PulsePay is sized as a serverless-first architecture rather than a single large VM, since its workload is short, bursty verification requests rather than a constantly-loaded service. The API layer runs on Cloud Functions sized conceptually at 2 vCPU / 4 GB, sufficient for OCR orchestration and request validation. The OCR and Error Level Analysis (ELA) module is the heaviest component and is planned to run on a Cloud Run container at roughly 4 vCPU / 8 GB where sustained image processing exceeds standard Cloud Function limits. Firestore is used as the managed database for verification records, and Cloud Storage holds raw screenshots, keeping both out of the compute tier's storage and memory budget. Local development uses a Type-2 hypervisor (VirtualBox) running a single prototype VM (2 vCPU, 4 GB RAM, 40 GB disk) to test the pipeline offline before any cloud billing begins. In the cloud pilot, cloud-managed virtualization is used throughout, so no hypervisor is administered directly. Snapshots are taken before every production deployment, schema change and evaluator demo, giving a fast rollback point; clones are used to test new fraud-scoring model versions and to create a disposable demo copy for presentations. The VM-vs-container decision favors managed/serverless services for the frontend, backend, database and storage, and reserves containers specifically for the AI/ELA module, where predictable CPU/memory matters more than fast cold starts. Overall storage is planned at roughly 40 GB for the app tier plus a growing Cloud Storage bucket (starting around 30 GB) for screenshot retention. This plan intentionally starts small — one prototype VM and serverless functions — with a clear path to containerizing the ML module and adding autoscaling as merchant volume grows.

Carry-Forward to CO2 AT2 and CO2 AT3

Output from CO2 AT1	How It Will Be Used Later
VM sizing values	Used as configuration target in CO2 AT2 practical evidence
Hypervisor/virtualization choice	Used to justify deployment environment
Snapshot/clone plan	Used as backup and testing evidence
VM/container comparison	Used in CO2 AT3 infrastructure design case
Final recommendation	Used as the capstone infrastructure baseline

GitHub Submission Checklist

- Completed workbook file.
- README with capstone title, sizing summary and final recommendation.
- Optional architecture diagram or table exported as image/PDF.
- Repository link submitted in Google Classroom.